Diffusion-Based Generation of Novel Pokemon with Conditional Evolution Modeling

Aaron Cyril John, Yugaank Kalia, Varen Maniktala

MSML 612 Final Report

Abstract

This work presents a conditional diffusion model for generating novel Pokemon-style artwork under structured controls for elemental types, evolution stage, visual style (sprite, Sugimori, or 3D render), and an optional previous-evolution reference image. We assemble a diffusion-ready training corpus of 6,373 labeled entries drawn from merged public asset packs, implement a FiLM-conditioned U-Net with 128x128 RGB image inputs, self-attention at coarse resolutions, and a dedicated previous-evolution image encoder. We document training, sampling, and evaluation against an unconditional diffusion baseline, while retaining the interim image-to-image GAN as a qualitative point of comparison. The report summarizes dataset curation and preprocessing, architectural and training choices, qualitative samples, quantitative FID/KID/ID evaluation, limitations, and reproducibility practices. The reference list and optional extended tables or figure grids are placed in an **appendix** after the main body.

1 Problem Statement

Pokemon-inspired character art must remain visually convincing while remaining faithful to the overall aesthetic: dual elemental types, staged evolution chains, and a consistent look-and-feel across the distinct official asset styles. Building a generative model for this setting is tricky because high-quality sources use incompatible filenames and variant naming (regional forms, Mega, Gigantamax, shiny palettes), labels such as evolution stage are not provided directly by any single upstream pack and the usable training sets are small compared to typical web-scale diffusion datasets.

The baseline for the final system is an **unconditional diffusion** model trained only on Pokemon images, with no labels, no evolution pairs, and no reference sprite. That model can learn the overall visual distribution, but it cannot control type, stage, style, or base-to-evolution progression. The objective of this final project is therefore a **conditional diffusion** model on a unified RGB dataset that folds heterogeneous conditioning (multi-hot types, stage, style, timestep, and optional previous-stage imagery) into a single control signal, supports multi-style training in one network, and allows a systematic comparison to the unconditional diffusion baseline on sample quality and conditioning adherence.

2 Related Work and Literature Review

Diffusion and score-based image models. Denoising diffusion probabilistic models (DDPMs) learn a reverse noising process for image generation and have become a standard alternative to GANs for high-fidelity synthesis [1]. Training and sampling recipes were refined by improved noise schedules and parameterizations [2], and diffusion models were later shown to be competitive with or superior to GANs on large-scale benchmarks when combined with guidance mechanisms [3]. Subsequent work explores latent-space diffusion for efficiency at higher resolutions [4] and broader design spaces for diffusion training objectives [5]. Classifier-free guidance [6] is a widely used technique for strengthening conditioning at inference time.

Backbones, conditioning, and image-to-image diffusion. Convolutional U-Net encoders and decoders remain the de facto spatial backbone for pixel-space diffusion [8]. Feature-wise linear modulation (FiLM) provides a lightweight mechanism to inject global vectors into convolutional blocks [9], which we apply at every residual block rather than only at the bottleneck. For paired image-to-image settings, Palette-style diffusion formulations translate between domains with diffusion losses [7], conceptually related to our earlier GAN translator though we ultimately

target unconditional and attribute-conditioned generation rather than deterministic source-to-target replay alone. Conditional GANs such as pix2pix [10] remain relevant baselines for paired translation quality.

Pokemon generation baselines. Prior Pokemon-generation projects mostly produce standalone Pokemon-like images. Generated-Pokemon trains an unconditional DDPM/DDIM model on 819 JPG images and does not condition on type, style, stage, or previous evolutions [16]. Wong's CS230 Pokemon generator uses StyleGAN2-ADA and a separate name-generation component, but does not use a diffusion objective or explicit evolution and attribute controls [17]. Our contribution is a structured, controllable, evolution-aware dataset and model rather than a standalone image generator.

Positioning. Our implementation follows public DDPM and U-Net practice, draws on improved diffusion and guidance ideas [2,3], and combines FiLM-style conditioning [9] with an additional CNN encoder for previous-evolution image context. Domain-specific contributions are the merged multi-style JSON dataset interface, dual-type multi-hot encoding, explicit masking for missing prior-stage images, and style/stage/type-conditioned sampling across sprite, Sugimori, and 3D domains.

3 Dataset and Preprocessing

The dataset combines three complementary visual sources and enriches each entry with structured Pokemon metadata:

- **SugimoriSprites**: official-style 2D Pokemon artwork used as high-quality reference art.
- **PokeSprite**: compact game-style sprite assets used as the primary target sprite representation.
- **ProjectPokemon / related render assets**: large 3D render images used as an additional visual source for cross-format alignment.

3.0.1 Table 1: Raw image sources

Dataset	Raw Images
SugimoriSprites	
PokeSprite	
ProjectPokemon	

3.0.2 Table 2: Raw dataset summary

Source	Files	Folders	Role
SugimoriSprites	1,848	1,025	High-quality 2D reference art
PokeSprite	2,853	905	Game-style sprite (primary target domain)
ProjectPokemon 3D	2,799	1,026	Alternate visual domain for cross-format use

3.1 Cross-format Mapping (GAN baseline era)

Our earlier GAN baseline required explicit source-to-target image pairs. We implemented mapping scripts that parsed filenames, normalized naming differences, handled form-specific cases (Mega, Gigantamax, regional variants), and produced CSV mappings. An important part of the curation effort was manual verification via page-based contact sheets for side-by-side inspection of mapped pairs, because naming conventions across sources are inconsistent and automatic matching alone is not reliable enough for a clean training set.

- `mapping_sugimori_to_sprite.csv`: **1,710** automatically generated Sugimori-to-sprite mappings across **905** Pokemon folders.
- `edited_mapping_sugimori_to_sprite.csv`: **977** manually edited and verified Sugimori-to-sprite mappings across **830** Pokemon folders.
- `mapping_3d_to_sprite.csv`: **2,676** 3D-to-sprite mappings across **905** Pokemon folders.

3.2 Diffusion-Ready Dataset (Final)

For the final conditional diffusion model, we moved from paired translation (source-style to sprite) to a unified generative dataset in which **every entry is a (target image, conditioning vector) pair**, optionally with a previous-evolution image to support evolution-chain conditioning. The per-entry schema is:

```
{
  "prev": "bulbasaur", "next": "ivysaur",
  "prev_sprite": "poke-data/PokeSprite/0001bulbasaur/bulbasaur.png",
  "next_sprite": "poke-data/PokeSprite/0002ivysaur/ivysaur.png",
  "types": ["grass", "poison"],
  "evolution_stage": "evo 1",
  "art_style": "sprite"
}
```

3.2.1 Table 3: Diffusion-ready mappings

Mapping File	Entries	Role
`safe_sprite_to_sprite_types_evolution.json`	1,995	Sprite -> Sprite (+ prev-evo image)
`safe_sugimori_to_sugimori_types_evolution.json`	1,674	Sugimori -> Sugimori (+ prev-evo image)
`safe_project_to_project_types_evolution.json`	2,704	3D -> 3D (+ prev-evo image)
Total (merged in `PokemonDiffusionDataset`)	**6,373**	Unified diffusion training set

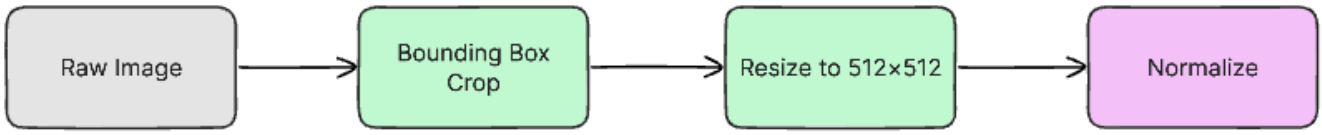
3.3 Curation Challenges

Building these mappings required solving problems that off-the-shelf datasets do not:

- **Cross-format name collisions.** The same Pokemon appears under incompatible filename conventions across sources (for example `0006 Charizard.png`, `charizard.png`, and `poke_capture_0006_*.png`). Resolved via normalized lookup against `poke-data/pokedex.json`.
- **Regional and cosmetic variants.** Names such as `meowth-galar`, `bulbasaur shiny`, Mega, and Gigantamax share base Pokedex IDs. These suffixes are stripped and tagged so that type lookup stays unambiguous.
- **Evolution-stage resolution.** No single source labels the evolution stage directly. We derive stage by **BFS over the `prev -> next` edges from chain roots**, with a **PokeAPI fallback** for entries not covered by our locally curated chains.

- **Art-style labelling.** Each mapping file is tagged with its visual domain (`sprite`, `sugimori`, `3d`) so that a single unified dataset drives **style-conditioned** sampling at inference time.

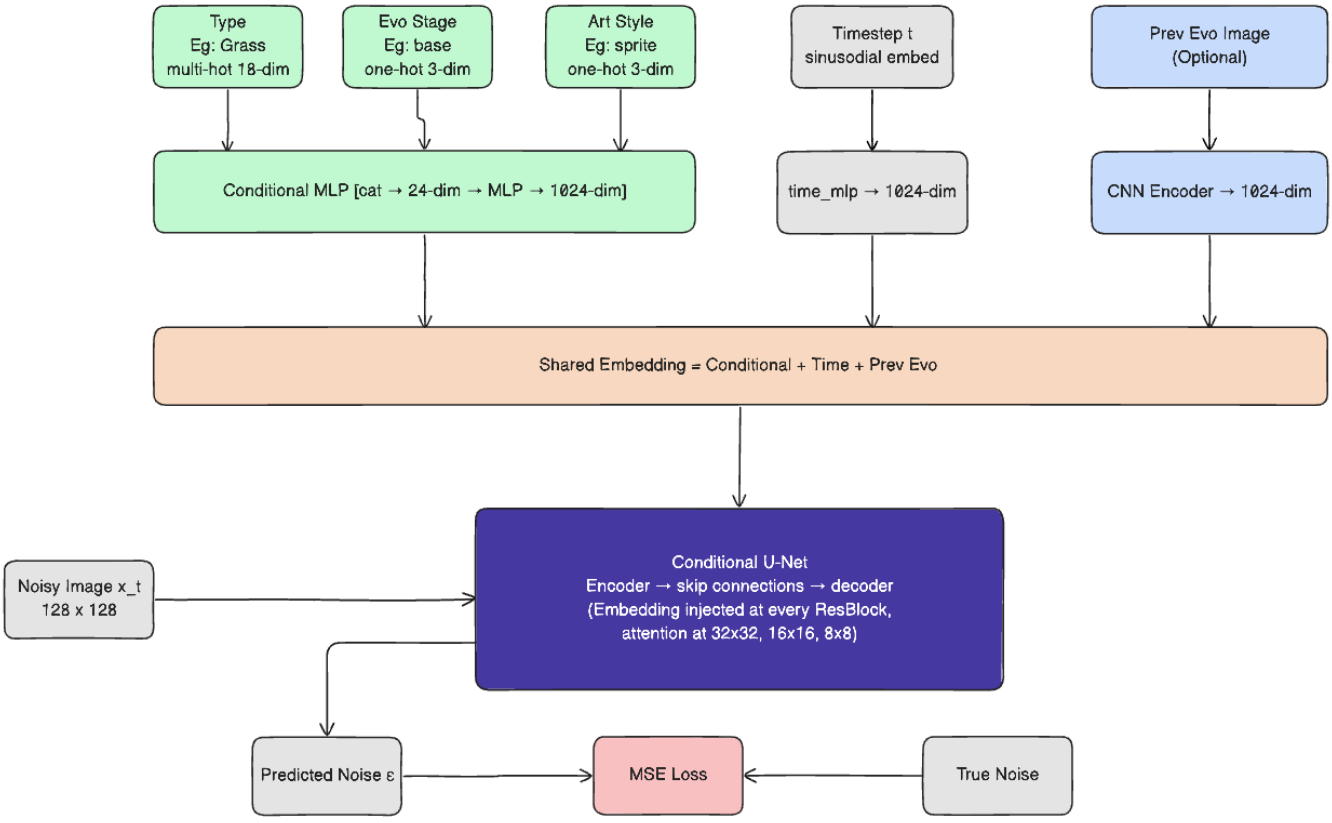
3.4 Pre-processing Pipeline



Filename normalization feeds into form-variant handling (Mega, Gigantamax, regional), which feeds into cross-format mapping generation, which is then enriched with `types` and `evolution_stage` (from local chains plus PokeAPI), and finally resized to **RGB 128x128** so that sprites, Sugimori artwork, and 3D renders share one consistent tensor representation.

4 Model Architecture and Implementation Details

4.1 Architecture overview



The model is a **conditional U-Net** with:

- **Base channel width 128**, channel multipliers `(1, 2, 2, 4)`, **2 ResBlocks per level**.
- **FiLM (gamma, beta) modulation at every ResBlock**, driven by a shared conditioning vector.
- **Self-attention at 32x32, 16x16, and 8x8** spatial resolutions, to capture long-range shape coherence that pure convolutions miss.
- **Dropout 0.1** inside ResBlocks for regularization on a small dataset.
- Trained at **128x128 RGB**.

4.2 Conditioning design

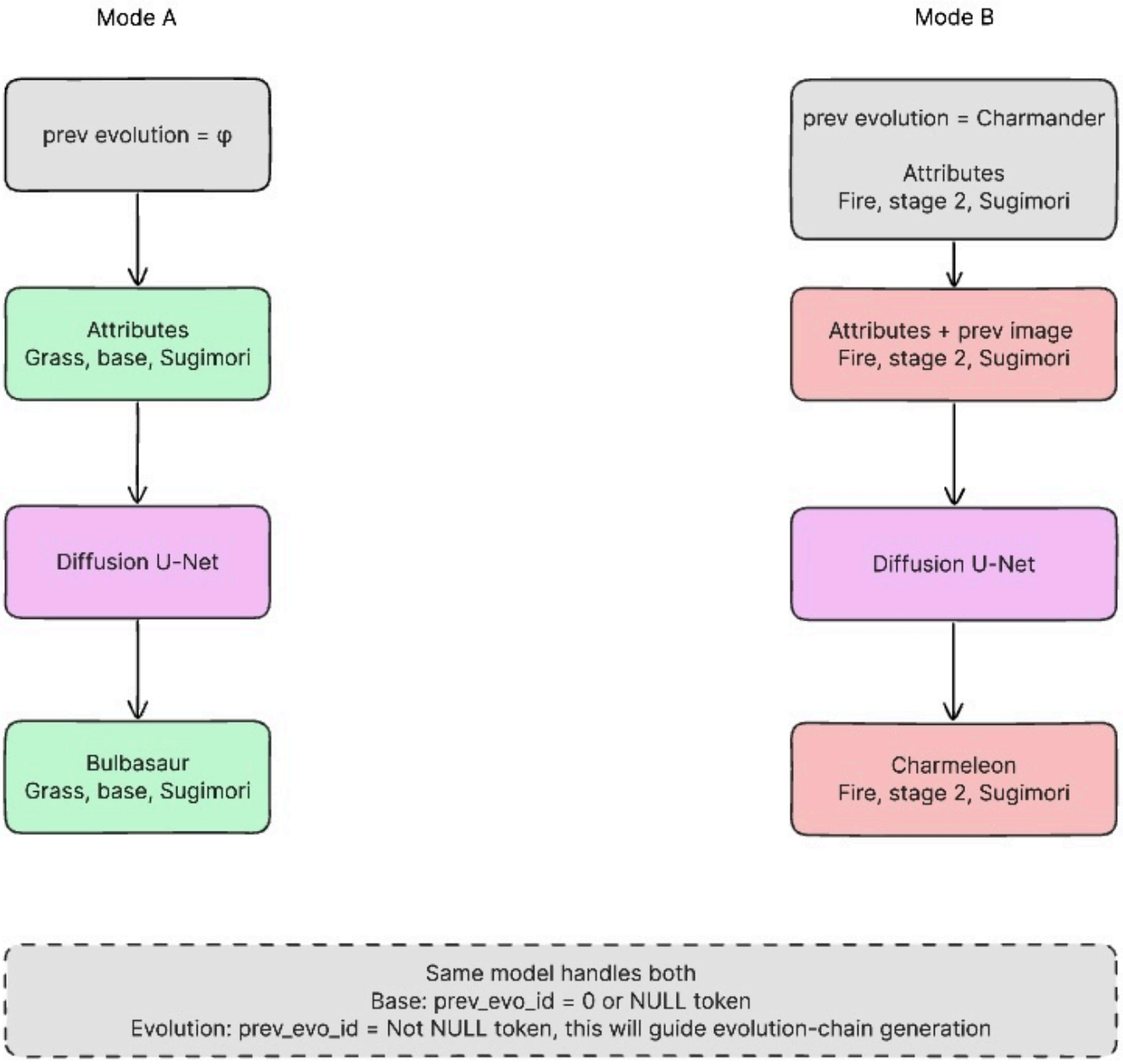
Five heterogeneous conditioning signals (categorical, set-valued, continuous, and image-valued) are fused into a single 128-dimensional control vector that drives FiLM modulation throughout the network.

4.2.1 Table 4: Conditioning inputs

Conditioning Input	Encoding	Why this encoding
Pokemon Type (e.g., Fire, Water)	**Multi-hot over 18 types** (supports dual-type)	One-hot would lose dual-typing entirely
Evolution Stage (base / evo1 / evo2)	One-hot over 3 stages	Discrete structural complexity control
Visual Style (`3d` / `sugimori` / `sprite`)	One-hot over 3 styles	Enables cross-domain sampling at inference
Previous Evolution Image	CNN encoder -> 128-d vector (masked if absent)	Base-stage Pokemon have no prior; mask token keeps batch shape valid
Timestep `t`	Sinusoidal embedding -> MLP	Standard DDPM timestep encoding

All embeddings are concatenated and fused by a `cond_combine` MLP into a single 128-dim vector, which drives **FiLM (gamma, beta) modulation** inside every ResBlock at every resolution, not just at the bottleneck. This places the conditioning signal close to every feature map rather than relying on a single injection point, and matches the design motivation of FiLM [9].

4.3 Training procedure



4.3.1 Table 5: Training setup

Component	Setting
Objective	MSE between predicted noise and true noise (standard DDPM)
Optimizer	AdamW, `lr = 2e-4`, `weight_decay = 1e-4`
LR schedule	Cosine annealing over 500 epochs
Noise schedule	Linear beta from `1e-4` to `0.02` over **1000 timesteps**

Component	Setting
EMA	Decay = 0.9999 on model weights (used for sampling)
Regularization	Gradient clipping at 1.0, dropout 0.1 inside ResBlocks
Batch size	64
Image size	128 x 128 RGB
Base channels / mults / blocks	128 / (1, 2, 2, 4) / 2 ResBlocks per level
Attention resolutions	32 x 32, 16 x 16, 8 x 8
Hardware	Google Colab G4 High-RAM GPU: NVIDIA RTX PRO 6000 Blackwell

4.3.2 Table 6: Training outcome

Component	Value
Parameters	Approximately 21M
Epochs completed	500
Wall-clock training time	Approximately 5 hours
Training curve	See Figure 1
Hardware	Google Colab G4 High-RAM GPU: NVIDIA RTX PRO 6000 Blackwell

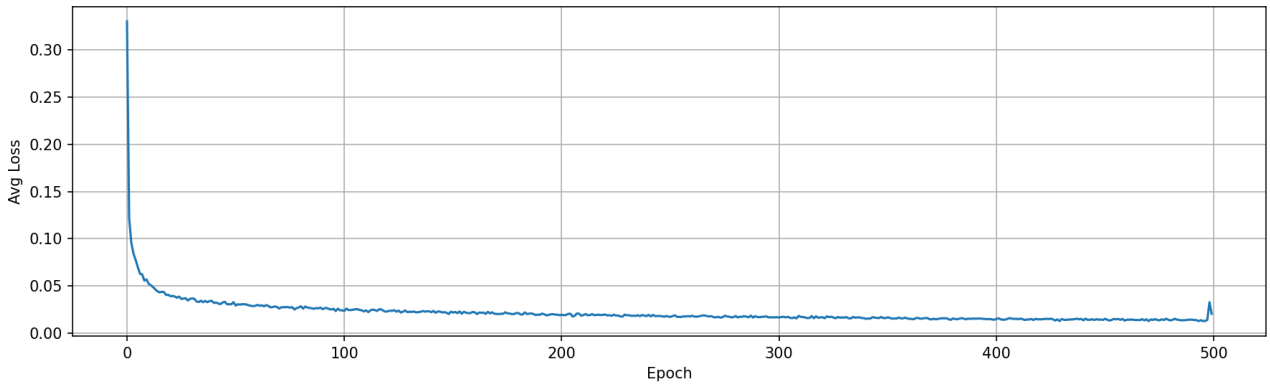


Figure 1: Training loss over 500 epochs for the unconditional diffusion baseline.

4.4 Software layout

Training and evaluation are driven from a small set of Python entry points with YAML configuration (`configs/final.yaml`), a single `PokemonDiffusionDataset` loader over the three merged JSON mapping files, and version-controlled train/validation index JSON for repeatable splits. Checkpoints store both raw and EMA weights and support `--resume`; sampling exposes flags for type, stage, style, and optional evolution-chain scripts. Further tooling and dependency pins are summarized in Section 8.

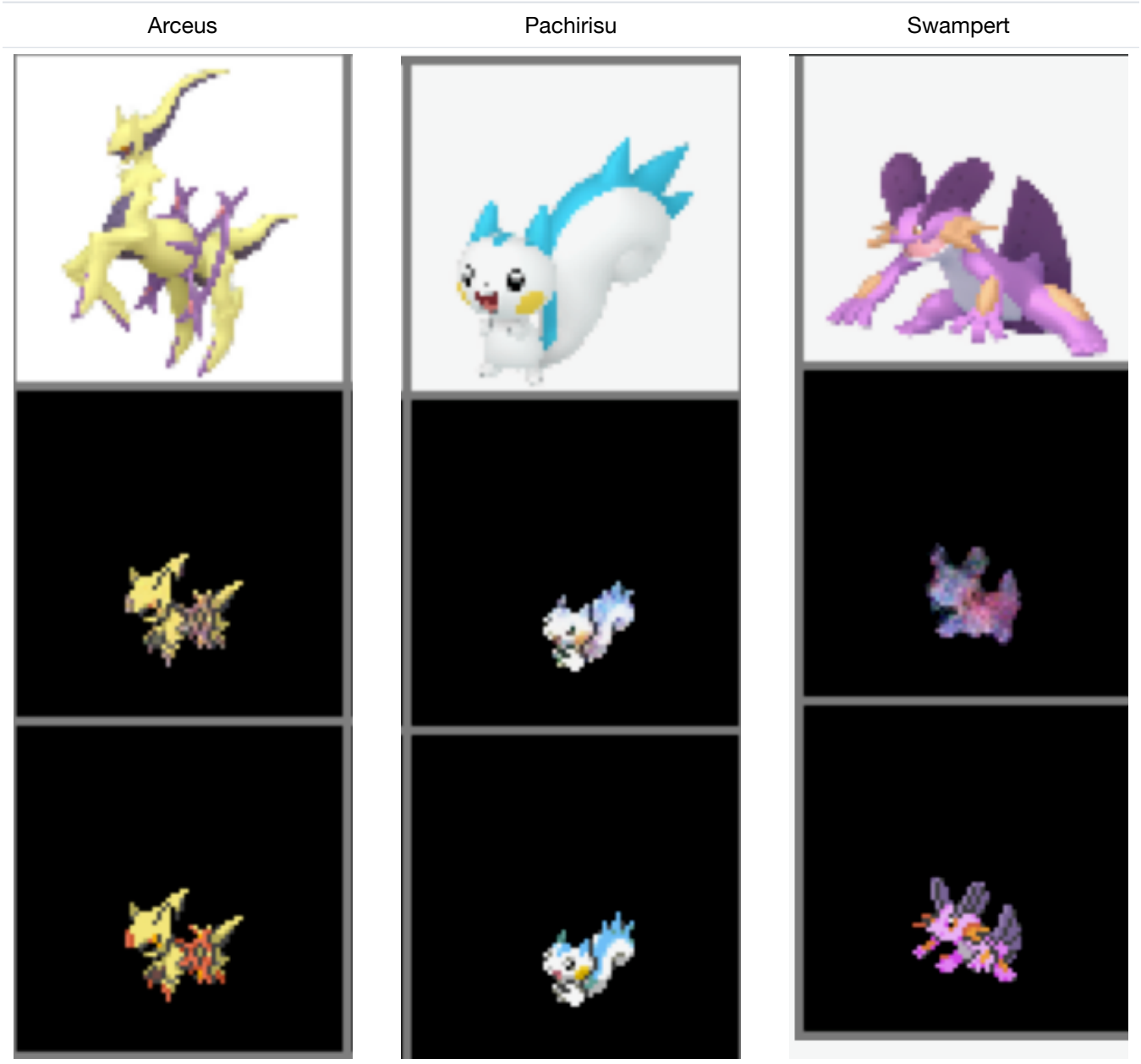
5 Evaluation Results

5.1 Qualitative results

5.1.1 Preliminary GAN baseline (from interim)

As a baseline, we trained a GAN-based image-to-image model that translates Sugimori/3D artwork into game-style sprites. The following figure shows three sample results; each column shows the input artwork (top), the GAN-predicted sprite (middle), and the original ground-truth sprite (bottom).

Figure 1. Preliminary GAN Results (Sugimori/3D -> Sprite)



The GAN captured broad color palettes and rough silhouettes but lacked fine detail and sharpness compared to the ground-truth sprites. The diffusion results below shift the comparison from paired translation to image generation, where structured conditioning provides direct control over type, style, and stage.

5.1.2 Unconditional diffusion samples

5.1.3 Type-conditioned samples

5.1.4 Style-conditioned samples

5.1.5 Evolution-chain generation

The implemented sampler supports evolution-chain generation by feeding a generated stage forward as the next stage's previous-evolution image. Qualitatively, these chains demonstrate the intended conditioning interface, but

3D samples Sprite samples

Figure 2: Unconditional diffusion samples from the image-only baseline.

3D Sprites

Figure 3: Type-conditioned diffusion samples in the 3D and sprite domains.

Figure 4: Style-conditioned samples spanning 3D, Sugimori, and sprite outputs.

the generated forms do not yet maintain enough visual coherence across stages to be presented as a final result figure.

5.1.6 Diffusion vs. GAN (paired comparison)

The GAN baseline remains useful as a qualitative reference for the limitations of paired translation, but the final quantitative comparison uses unconditional diffusion as the baseline because it isolates the effect of adding structured conditioning to the same diffusion family.

5.2 Quantitative evaluation, runtime, and ablations

5.2.1 Table 8: Evaluation metrics

Metric	Description	What it measures
FID (Frechet Inception Distance)	Distribution distance between real and generated images; lower is better	Realism and distribution match
KID (Kernel Inception Distance)	FID-like distribution metric that is less biased for smaller sample counts	Small-sample realism and diversity
ID (Inception Distance)	Average pairwise distance across generated samples	Sample diversity
Qualitative Review	Side-by-side visual inspection with attribute adherence	Conditioning adherence

5.2.2 Quantitative scores

5.2.3 Table 9: Quantitative comparison

Metric	Unconditional Diffusion Baseline	Conditional Diffusion (Single Style)	Conditional Diffusion (Multiple Styles)
FID (lower is better)	123	**77**	202
KID (lower is better)	0.132 ± 0.002	**0.061 ± 0.0017**	0.231 ± 0.033
ID (higher is better)	15.44	**16.23**	17.39

The single-style conditional model improves both FID and KID over the unconditional diffusion baseline, while the multi-style model increases the diversity-oriented ID score at the cost of weaker distributional match. This matches the qualitative observation that multi-style training is harder because one network must cover sprite, Sugimori, and 3D visual domains at once.

5.2.4 Runtime and efficiency

Performance is not just sample quality; the rubric also credits running time and practical cost.

5.2.5 Table 10: Runtime and efficiency

Measurement	Unconditional Diffusion Baseline	Diffusion (ours)
Parameters	Approximately 21M	Approximately 21M
Training hardware	Google Colab G4 High-RAM GPU	Google Colab G4 High-RAM GPU
Wall-clock training time	Approximately 5 hours for 500 epochs	Approximately 5 hours for 500 epochs
Sampling steps	1000 DDPM steps; DDIM used for faster previews	1000 DDPM steps; DDIM used for faster previews
Input resolution	128 x 128 RGB	128 x 128 RGB

Both diffusion variants use the same broad U-Net scale, so the reported runtime comparison primarily reflects the additional conditioning inputs rather than a wholesale change in model family.

5.2.6 Ablation study

5.2.7 Table 11: Ablations

Experiment	What it tests	FID	Notes
Unconditional vs. attribute-conditioned	Effect of type/stage/style on quality	123 vs. 77	Single-style conditioning improves FID
Single-style vs. multi-style joint training	Cross-domain transfer	77 vs. 202	Multi-style training increases task difficulty
Unconditional vs. multi-style conditional	Diversity under broader conditioning	123 vs. 202	Multi-style model has higher ID but worse FID
EMA weights at sampling	Sample stability	Used	EMA checkpoint used for stable samples

The ablation table summarizes the comparisons completed for the final presentation. A cleaner with/without previous-evolution-reference experiment remains future work because the current evolution-chain outputs lack enough stage-to-stage coherence for a fair quantitative claim.

6 Insights, Limitations, and Future Scope

6.1 Insights

What worked well: The unified multi-source dataset (about 6.4K entries) with clean `types`, `stage`, and `style` labels made it possible to train a single model that generalizes across sprite, Sugimori, and 3D styles. FiLM conditioning integrates cleanly with the U-Net, with no architectural surprises. The single-style conditional model achieved better FID and KID than the unconditional diffusion baseline, showing that structured labels are useful rather than only decorative.

What was hard: Cross-format name normalization (shiny, mega, gigantamax, regional variants) required careful hand-written rules; relying on filenames alone was insufficient. Evolution-stage resolution had to be implemented with BFS over `prev -> next` edges plus a PokeAPI fallback because no single source labels stage directly. Preprocessing also took a large fraction of the project because naming, orientations, and visual structures were inconsistent across art styles. Finally, the dataset is small relative to typical diffusion training runs, which made careful augmentation and the EMA model copy important for stable sampling.

Takeaway: Relative to the unconditional diffusion baseline, the conditional diffusion formulation is a better match for a structured, low-data, multi-domain Pokemon setting: iterative denoising remains stable, attribute conditioning is integrated at every residual block through a shared FiLM vector, and one model can be trained over all three visual styles.

6.2 Limitations

- Trained at **128x128 RGB**; fine sprite detail is still bounded by resolution.
- Only 3 styles and 3 evolution stages are modeled; real Pokemon have far more visual variants (shiny, mega, regional forms) that we do not yet condition on explicitly.
- No text conditioning; the model cannot follow natural-language prompts describing abilities or lore.
- The code supports evolution-chain sampling, but the generated chains do not yet maintain enough visual coherence across stages.

6.3 Future scope

- **Scale up resolution** with a latent diffusion or super-resolution stage.
- **Classifier-free guidance** via conditioning dropout for stronger attribute adherence at sampling time [6].
- **Text conditioning** using a pretrained text encoder (e.g., CLIP) for descriptive generation.
- **Full evolution-chain consistency loss** that ties together all generated stages jointly rather than one stage at a time.
- **RGBA masking** or an alpha-aware post-processing stage to recover cleaner transparent sprite outputs from RGB generation.

7 Contribution of Each Team Member

- **Aaron Cyril John:** Led final report and slide integration, curated evaluation figures and metrics, reconciled preprocessing and training documentation, and supported model training and result analysis.
- **Yugaank Kalia:** Contributed to dataset curation, mapping verification, preprocessing design, model experimentation, and presentation materials.
- **Varen Maniktala:** Contributed to dataset alignment, architecture review, training/evaluation support, and final presentation/report preparation.

8 Tools and Libraries Used

8.1 Core libraries and stack

The implementation is written in **Python 3** with **PyTorch** and **torchvision** for tensor computation, optimizers, and image I/O/transforms. Configuration is loaded from **YAML** files (`configs/final.yaml`). Numerical utilities rely on **NumPy**; plotting and contact-sheet style figures for curation use **Matplotlib** where applicable. Optional preprocessing utilities in the repository may additionally use **Pandas** and **Pillow**. The GAN-era exploratory stack referenced **Streamlit** for lightweight inspection in some preprocessing paths. External metadata is supplemented via the **PokeAPI** HTTP API [12]. Public references for diffusion tooling include the Hugging Face **Diffusers** library as related reading [11], though the course model code is implemented directly in PyTorch.

8.2 Compute and reproducibility

Training and timing numbers in this report were obtained on a **Google Colab G4 High-RAM GPU with an NVIDIA RTX PRO 6000 Blackwell** and on HPC/local CUDA environments where noted. The repository pins dependency versions in `requirements.txt` (paths may vary by subpackage; see repository root and `preprocessing/requirements.txt`).

8.2.1 Table 7: Reproducibility practices

Concern	How it is handled
Deterministic runs	Fixed seeds for `torch`, `numpy`, `random`; deterministic `DataLoader` workers
Dataset splits	Train / val indices saved to JSON, identical across every experiment
Configuration	YAML-driven (`configs/final.yaml`); all hyperparameters live outside the code
Data loading	Single `PokemonDiffusionDataset` class unifying all three mapping files
Entry point	`python main.py` supports training via `--train` and inference via `--inference`
Environment	Pinned `requirements.txt`; tested on Google Colab G4 and HPC/local CUDA environments
Checkpoints	EMA + raw weights saved every N epochs; resumable via `--resume`
Sampling	`python sample.py --ckpt ... --type fire --stage base --style sprite`

The mappings, splits, training run, and sampling grids shown in this report are produced by scripts checked into the repository so that results can be regenerated without undocumented manual steps.

Course formatting (page budget). The preceding sections—from the abstract through this tools section—constitute the **main report**. Per course instructions, keep this main portion to **at most 20 pages** when compiled (title page counts toward that total unless your instructor specifies otherwise). The main portion ends on page 13. If the main body grows too long, move bulky tables, extra result grids, contact sheets, or per-metric breakdowns into Appendix B below rather than expanding the core narrative.

A References

[1] Ho, J., Jain, A., and Abbeel, P. *Denoising Diffusion Probabilistic Models*. NeurIPS, 2020. https://arxiv.org/abs/2006.11239

[2] Nichol, A., and Dhariwal, P. *Improved Denoising Diffusion Probabilistic Models*. ICML, 2021. https://arxiv.org/abs/2102.09672

[3] Dhariwal, P., and Nichol, A. *Diffusion Models Beat GANs on Image Synthesis*. NeurIPS, 2021. https://arxiv.org/abs/2105.05233

[4] Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. *High-Resolution Image Synthesis with Latent Diffusion Models*. CVPR, 2022. https://arxiv.org/abs/2112.10752

[5] Karras, T., Aittala, M., Aila, T., and Laine, S. *Elucidating the Design Space of Diffusion-Based Generative Models*. NeurIPS, 2022. https://arxiv.org/abs/2206.00364

[6] Ho, J., and Salimans, T. *Classifier-Free Diffusion Guidance*. NeurIPS Workshop, 2021. https://arxiv.org/abs/2207.12598

[7] Saharia, C., Chan, W., Saxena, S., et al. *Palette: Image-to-Image Diffusion Models*. SIGGRAPH, 2022. https://arxiv.org/abs/2111.05826

[8] Ronneberger, O., Fischer, P., and Brox, T. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. MICCAI, 2015. https://arxiv.org/abs/1505.04597

[9] Perez, E., Strub, F., de Vries, H., Dumoulin, V., and Courville, A. *FiLM: Visual Reasoning with a General Conditioning Layer*. AAAI, 2018. https://arxiv.org/abs/1709.07871

[10] Isola, P., Zhu, J.-Y., Zhou, T., and Efros, A. A. *Image-to-Image Translation with Conditional Adversarial Networks (pix2pix)*. CVPR, 2017. https://arxiv.org/abs/1611.07004

[11] Hugging Face. *Diffusers: State-of-the-art diffusion models for image and audio generation in PyTorch*. https://github.com/huggingface/diffusers

[12] PokeAPI. https://pokeapi.co/

[13] msikma. *pokesprite*. https://github.com/msikma/pokesprite

[14] Project Pokemon. Sprite and asset resources. https://projectpokemon.org/

[15] Pokemon Database. National Pokedex and image resources. https://pokemondb.net/pokedex/national

[16] Followb1ind1y. *Generated-Pokemon: New Pokemon Generation using Diffusion Models*. https://github.com/Followb1ind1y/Generated-Pokemon

[17] Wong, R. *Pokemon Generator*. Stanford CS230, 2021. https://cs230.stanford.edu/projects_fall_2021/reports/103146257.pdf

B Supplementary tables and figures

Use this appendix section for material that supports the main report but need not appear in the 20-page core: for example full-page sampling grids (unconditional, type- or style-conditioned strips, evolution chains, GAN-vs-diffusion comparisons), extended quantitative tables (per-style FID, full ablation grids), or additional mapping contact sheets. The main body should retain only the figures and tables needed to follow the argument; move overflow content here and cite it explicitly from the main text (e.g., “see Appendix B”).